```
================================================================================
 AI POWERED WORKFORCE MANAGEMENT SYSTEM
 Complete Build Guide (From Scratch) - Final Year Project
 Desktop Software for Windows
================================================================================

TABLE OF CONTENTS
  1.  Project Overview
  2.  Final Tech Stack
  3.  Software to Install (Setup Checklist)
  4.  System Architecture
  5.  Project Folder Structure
  6.  Roles and Permissions
  7.  Database Design (Tables)
  8.  Core Modules (Detailed)
  9.  REST API List
  10. AI Feature 1: Face Recognition Attendance
  11. AI Feature 2: Smart Shift Scheduling
  12. AI Feature 3: Attrition Prediction
  13. AI Feature 4: HR Chatbot (Local Ollama)
  14. Desktop App Packaging (Electron)
  15. One-Click Start Script
  16. Week-by-Week Build Plan
  17. Testing Plan
  18. Report / Documentation Structure
  19. Demo Plan
  20. Risks and Tips
  21. What To Do Right Now (Day 1)



================================================================================
1. PROJECT OVERVIEW
================================================================================

Name: AI Powered Workforce Management System

Goal:
  A desktop software that lets a company manage employees, attendance,
  leave, tasks, shifts and payroll, with AI features that automate work
  and give smart insights.

Problem it solves:
  - Manual attendance is slow and can be faked (buddy punching).
```

```
  - Shift rosters are built by hand and cause conflicts.
  - HR cannot see which employees are likely to quit.
  - Employees keep asking HR the same questions (leave balance, policy).

AI features (4):
  1. Face recognition attendance
  2. Smart shift scheduling (optimization)
  3. Attrition (resignation) risk prediction (machine learning)
  4. HR chatbot (local LLM, works offline)

Key design decision:
  Everything runs LOCALLY. No paid API, no internet needed for the demo.


================================================================================
2. FINAL TECH STACK
================================================================================

Backend (main system)
  - Java 17 (or 21)
  - Spring Boot 3.x
  - Spring Web (REST APIs)
  - Spring Security + JWT (login and roles)
  - Spring Data JPA + Hibernate (database access)
  - Validation (Jakarta Validation)
  - Lombok (less boilerplate)
  - Maven (build tool)

Database
  - PostgreSQL 16 (recommended) or MySQL 8

Frontend (desktop UI)
  - React (Vite)
  - Axios (calls backend)
  - React Router (pages)
  - Tailwind CSS or Material UI (design)
  - Recharts or Chart.js (dashboard charts)

Desktop wrapper
  - Electron
  - electron-builder (creates Windows installer .exe)

AI service (Python microservice)
```

```
  - Python 3.10 or 3.11
  - FastAPI + Uvicorn (small API server)
  - scikit-learn, pandas, joblib (attrition model)
  - face_recognition (dlib) OR DeepFace (face matching)
  - OpenCV (camera, image handling)

Scheduling
  - Google OR-Tools (CP-SAT solver), Java version, inside Spring Boot

Chatbot
  - Ollama (runs local models)
  - Model: llama3.2:3b or phi3:mini (fits your 16 GB RAM laptop)

Tools
  - VS Code or IntelliJ IDEA Community (Java)
  - Postman (test APIs)
  - Git + GitHub (version control, backup)
  - DBeaver or pgAdmin (view database)


================================================================================
3. SOFTWARE TO INSTALL (SETUP CHECKLIST)
================================================================================

Install in this order and tick them off:

  [ ] JDK 17 (Temurin / Adoptium)
  [ ] IntelliJ IDEA Community (or VS Code + Java extensions)
  [ ] Maven (IntelliJ includes it, so optional)
  [ ] PostgreSQL 16 + pgAdmin (remember your password)
  [ ] Node.js 20 LTS (includes npm)
  [ ] Python 3.10 or 3.11 (tick "Add to PATH" in installer)
  [ ] Git
  [ ] Postman
  [ ] Ollama (from ollama.com)
  [ ] Visual Studio Build Tools (C++ workload) - needed for dlib if you
      use face_recognition on Windows
  [ ] CMake (needed for dlib)

Verify installs (open Command Prompt):
  java -version
  node -v
  npm -v
```

```
  python --version
  git --version
  psql --version
  ollama --version

Pull the chatbot model (needs internet once):
  ollama pull llama3.2:3b

NOTE ON FACE LIBRARY:
  Installing dlib / face_recognition on Windows is the most painful setup
  step. If it fails after 1 hour, switch to DeepFace
  (pip install deepface) which installs easily and does the same job
  for your project. Don't waste days on it.


================================================================================
4. SYSTEM ARCHITECTURE
================================================================================

                +---------------------------+
                |   Electron Desktop App    |
                |   (React UI inside)       |
                +-------------+-------------+
                              |
                      HTTP (localhost:8080)
                              |
                +-------------v-------------+
                |   Spring Boot Backend     |
                |  - Auth / JWT             |
                |  - Core modules           |
                |  - OR-Tools scheduling    |
                +----+-----------+---------+
                     |           |         \
                     |           |          \
          +----------v--+   +-----v------+   +v-----------------+
          | PostgreSQL   |   | Python AI  |   | Ollama (LLM)     |
          | Database     |   | FastAPI    |   | localhost:11434  |
          |              |   | :8000      |   |                  |
          +--------------+   | - face     |   +------------------+
                             | - attrition|
                             +------------+


Ports:
```

```
  Spring Boot  -> 8080
  Python AI    -> 8000
  Ollama       -> 11434
  PostgreSQL   -> 5432
  React (dev)  -> 5173 (only while developing)

Flow example (face attendance):
  1. Employee opens Check-In page, webcam shows in the React UI.
  2. React captures a photo and sends it to Spring Boot.
  3. Spring Boot forwards the image to the Python AI service.
  4. Python compares it with stored face encodings, returns employee ID
     and confidence.
  5. Spring Boot saves the attendance record and returns success.


================================================================================
5. PROJECT FOLDER STRUCTURE
================================================================================

workforce-system/
|
|-- backend/                      (Spring Boot)
|   |-- pom.xml
|   |-- src/main/java/com/workforce/
|   |   |-- WorkforceApplication.java
|   |   |-- config/               (security, CORS, beans)
|   |   |-- security/             (JWT filter, util, user details)
|   |   |-- controller/           (REST controllers)
|   |   |-- service/              (business logic)
|   |   |-- repository/           (JPA repositories)
|   |   |-- entity/               (database tables as classes)
|   |   |-- dto/                  (request/response objects)
|   |   |-- scheduling/           (OR-Tools shift solver)
|   |   |-- ai/                   (clients for Python + Ollama)
|   |   |-- exception/            (global error handling)
|   |-- src/main/resources/
|       |-- application.properties
|
|-- frontend/                     (React + Electron)
|   |-- package.json
|   |-- vite.config.js
|   |-- electron/
|   |   |-- main.js
```

```
|   |   |-- preload.js
|   |-- src/
|       |-- api/                  (axios setup)
|       |-- components/
|       |-- pages/                (Login, Dashboard, Employees, ...)
|       |-- context/              (auth state)
|       |-- App.jsx
|
|-- ai-service/                   (Python)
|   |-- main.py                   (FastAPI app)
|   |-- face_service.py
|   |-- attrition_service.py
|   |-- train_attrition.py
|   |-- data/
|   |   |-- WA_Fn-UseC_-HR-Employee-Attrition.csv
|   |-- models/
|   |   |-- attrition_model.pkl
|   |-- faces/                    (stored face images or encodings)
|   |-- requirements.txt
|
|-- scripts/
|   |-- start-all.bat
|
|-- docs/                         (report, diagrams, screenshots)
|-- README.md


========================================================================
6. ROLES AND PERMISSIONS
========================================================================

ADMIN
  - Everything: manage users, roles, departments, system settings
  - View all reports and AI insights

HR
  - Manage employees, payroll, leave approvals
  - View attrition risk, run shift scheduling
  - Manage company policies for the chatbot

MANAGER
  - View own team
  - Assign tasks, approve team leave
```

```
  - View team attendance and shifts

EMPLOYEE
  - View own profile, attendance, payslips
  - Apply for leave, view tasks, view own shifts
  - Use chatbot, face check-in

Implementation: Spring Security with @PreAuthorize("hasRole('HR')") on
controller methods, role stored in the users table, embedded in the JWT.


================================================================================
7. DATABASE DESIGN (TABLES)
================================================================================

users
  id (PK), email (unique), password_hash, role (ADMIN/HR/MANAGER/EMPLOYEE),
  employee_id (FK), is_active, created_at

departments
  id (PK), name, description, manager_id (FK -> employees)

employees
  id (PK), employee_code (unique), first_name, last_name, email, phone,
  gender, date_of_birth, address, hire_date, job_title, department_id (FK),
  manager_id (FK -> employees), base_salary, status (ACTIVE/RESIGNED),
  photo_path, skills (text or separate table)

skills (optional, for scheduling)
  id (PK), name
employee_skills
  employee_id (FK), skill_id (FK)

attendance
  id (PK), employee_id (FK), date, check_in_time, check_out_time,
  method (FACE/MANUAL), confidence_score, status (PRESENT/LATE/ABSENT),
  work_hours

face_data
  id (PK), employee_id (FK), encoding (text/blob), image_path, created_at

leave_types
  id (PK), name (Annual/Sick/Casual), days_per_year
```

```
leave_requests
  id (PK), employee_id (FK), leave_type_id (FK), start_date, end_date,
  reason, status (PENDING/APPROVED/REJECTED), approved_by (FK),
  created_at

leave_balances
  id (PK), employee_id (FK), leave_type_id (FK), year, remaining_days

tasks
  id (PK), title, description, assigned_to (FK), assigned_by (FK),
  priority (LOW/MEDIUM/HIGH), status (TODO/IN_PROGRESS/DONE),
  due_date, created_at

shifts
  id (PK), name (Morning/Evening/Night), start_time, end_time,
  required_staff, required_skill_id (optional)

shift_assignments
  id (PK), employee_id (FK), shift_id (FK), date

employee_availability
  id (PK), employee_id (FK), day_of_week, available (boolean)

payroll
  id (PK), employee_id (FK), month, year, base_salary, bonus, deductions,
  tax, net_salary, generated_at

attrition_predictions
  id (PK), employee_id (FK), risk_score (0-1), risk_level, top_factors,
  predicted_at

policies (for chatbot)
  id (PK), title, content, updated_at

chat_history (optional)
  id (PK), user_id (FK), question, answer, created_at

audit_logs (optional, nice to have)
  id (PK), user_id, action, details, created_at

Tip: Draw the ER diagram in draw.io or dbdiagram.io for your report.
```

```
================================================================================
8. CORE MODULES (DETAILED)
================================================================================

8.1 Authentication
  - Login with email + password
  - Passwords hashed with BCrypt
  - Backend returns a JWT token, frontend stores it and sends it in the
    Authorization header: Bearer <token>
  - Auto logout when token expires

8.2 Employee Management
  - Add, edit, view, deactivate employees
  - Search and filter by name, department, status
  - Upload profile photo
  - View employee full profile page

8.3 Department Management
  - CRUD departments, assign a manager

8.4 Attendance
  - Check-in and check-out (face or manual fallback)
  - Late detection (e.g. after 9:15 AM = LATE)
  - Work hours calculated automatically
  - Monthly attendance report per employee and per department
  - Export to CSV

8.5 Leave Management
  - Employee applies (type, dates, reason)
  - Manager/HR approves or rejects
  - Leave balance auto-updated on approval
  - Calendar view of who is on leave

8.6 Task Management
  - Manager creates and assigns tasks
  - Employee updates status
  - Kanban-style board (To Do / In Progress / Done)
  - Overdue highlighting

8.7 Payroll
  - Salary = base + bonus - deductions - tax
  - Deductions from unpaid leave and lateness (simple rule)
```

```
  - Generate monthly payslips
  - Download payslip as PDF (use OpenPDF or iText library)

8.8 Dashboard
  - Admin/HR: total employees, present today, on leave, open tasks,
    attendance trend chart, department headcount pie chart,
    high attrition risk list
  - Employee: my attendance, my leave balance, my tasks, my shifts

8.9 Notifications (optional)
  - In-app notifications for leave approval, new task, new shift

8.10 Audit log (optional)
  - Record who changed what (good marks for security)


================================================================================
9. REST API LIST
================================================================================

AUTH
  POST   /api/auth/login
  GET    /api/auth/me

EMPLOYEES
  GET    /api/employees
  GET    /api/employees/{id}
  POST   /api/employees
  PUT    /api/employees/{id}
  DELETE /api/employees/{id}

DEPARTMENTS
  GET    /api/departments
  POST   /api/departments
  PUT    /api/departments/{id}

ATTENDANCE
  POST   /api/attendance/check-in            (manual)
  POST   /api/attendance/check-out
  POST   /api/attendance/face-check-in       (image upload)
  GET    /api/attendance?employeeId=&month=
  GET    /api/attendance/report

```

```
FACE REGISTRATION
  POST   /api/faces/register/{employeeId}    (image upload)

LEAVE
  POST   /api/leaves
  GET    /api/leaves/my
  GET    /api/leaves/pending
  PUT    /api/leaves/{id}/approve
  PUT    /api/leaves/{id}/reject
  GET    /api/leaves/balance

TASKS
  POST   /api/tasks
  GET    /api/tasks/my
  PUT    /api/tasks/{id}/status

SHIFTS
  POST   /api/shifts/generate?startDate=&endDate=
  GET    /api/shifts/roster?week=
  PUT    /api/shifts/assignments/{id}

PAYROLL
  POST   /api/payroll/generate?month=&year=
  GET    /api/payroll/my
  GET    /api/payroll/{id}/pdf

ATTRITION
  POST   /api/ai/attrition/run
  GET    /api/ai/attrition/results

CHATBOT
  POST   /api/ai/chat                        body: { "question": "..." }

DASHBOARD
  GET    /api/dashboard/summary

PYTHON AI SERVICE (internal, called only by Spring Boot)
  POST   http://localhost:8000/face/register
  POST   http://localhost:8000/face/recognize
  POST   http://localhost:8000/attrition/predict


================================================================================
```

```
10. AI FEATURE 1: FACE RECOGNITION ATTENDANCE
================================================================================

How it works
  Register: employee photo -> converted to a 128-number "face encoding" ->
            saved in the database.
  Check-in: webcam photo -> new encoding -> compared with all stored
            encodings -> closest match under a threshold = that employee.

Python service (face_service.py) idea:

    import face_recognition
    import numpy as np

    def get_encoding(image_path):
        img = face_recognition.load_image_file(image_path)
        encs = face_recognition.face_encodings(img)
        if len(encs) != 1:
            return None          # no face or multiple faces
        return encs[0]

    def recognize(image_path, known_encodings, known_ids, tolerance=0.5):
        enc = get_encoding(image_path)
        if enc is None:
            return None
        distances = face_recognition.face_distance(known_encodings, enc)
        best = int(np.argmin(distances))
        if distances[best] <= tolerance:
            confidence = 1 - distances[best]
            return known_ids[best], confidence
        return None

FastAPI endpoints (main.py):

    from fastapi import FastAPI, UploadFile, File

    app = FastAPI()

    @app.post("/face/register")
    async def register(employee_id: int, file: UploadFile = File(...)):
        # save image, compute encoding, store in memory + return encoding
        ...

```

```
    @app.post("/face/recognize")
    async def recognize_face(file: UploadFile = File(...)):
        # return {"employee_id": 12, "confidence": 0.91} or {"match": false}
        ...

Frontend webcam capture (React):
  - Use navigator.mediaDevices.getUserMedia({ video: true })
  - Show video in a <video> tag
  - On button click, draw a frame to <canvas>, convert to a Blob, send as
    multipart/form-data to /api/attendance/face-check-in

Important details
  - Register 3 to 5 photos per employee (different angles/lighting) for
    better accuracy.
  - Reject check-in if more than one face is found.
  - Prevent duplicate check-ins (one check-in per day, then check-out).
  - Anti-spoof (bonus): simple blink detection or ask user to turn head.
    Mention in report as "future improvement" if you skip it.
  - Keep a manual check-in fallback for HR when the camera fails.
  - Privacy: store encodings, not raw images, if possible. Mention this
    in your report (data protection point).

Metrics to report: accuracy on your test set (e.g. 20 people, 10 tries
each), false accept rate, false reject rate.


================================================================================
11. AI FEATURE 2: SMART SHIFT SCHEDULING
================================================================================

Goal
  Automatically create a weekly roster that covers all shifts, respects
  availability and rules, and shares work fairly.

Technique: Constraint Programming with Google OR-Tools CP-SAT.

Maven dependency:

    <dependency>
      <groupId>com.google.ortools</groupId>
      <artifactId>ortools-java</artifactId>
      <version>9.10.4067</version>
    </dependency>
```

```
    (check Maven Central for the latest version)

Model:
  Variable: x[e][d][s] = 1 if employee e works shift s on day d, else 0

Hard constraints (must hold):
  1. Each shift needs exactly (or at least) required_staff people.
  2. An employee works at most 1 shift per day.
  3. Employee only works days they are available / not on approved leave.
  4. Max shifts per week per employee (e.g. 5).
  5. No night shift followed by a morning shift next day (rest rule).
  6. If a shift needs a skill, at least N assigned employees have it.

Soft goals (optimize):
  - Spread shifts evenly (minimize difference between most and least
    shifts assigned).
  - Respect employee shift preferences where possible.

Java sketch:

    CpModel model = new CpModel();
    Literal[][][] x = new Literal[numEmp][numDays][numShifts];
    for (int e = 0; e < numEmp; e++)
      for (int d = 0; d < numDays; d++)
        for (int s = 0; s < numShifts; s++)
          x[e][d][s] = model.newBoolVar("x_" + e + "_" + d + "_" + s);

    // each shift covered
    for (int d = 0; d < numDays; d++)
      for (int s = 0; s < numShifts; s++) {
        LinearExprBuilder sum = LinearExpr.newBuilder();
        for (int e = 0; e < numEmp; e++) sum.add(x[e][d][s]);
        model.addEquality(sum, required[s]);
      }

    // at most one shift per day per employee
    for (int e = 0; e < numEmp; e++)
      for (int d = 0; d < numDays; d++) {
        LinearExprBuilder sum = LinearExpr.newBuilder();
        for (int s = 0; s < numShifts; s++) sum.add(x[e][d][s]);
        model.addLessOrEqual(sum, 1);
      }

```

```
    CpSolver solver = new CpSolver();
    CpSolverStatus status = solver.solve(model);
    // if status is OPTIMAL or FEASIBLE, read x values and save to
    // shift_assignments

Frontend
  - Weekly grid: rows = employees, columns = days, cells = shift names
  - "Generate Roster" button, then HR can manually drag/edit
  - Colour-code shifts

Report points: show a before/after comparison (manual roster with
conflicts vs. generated roster with zero conflicts, time taken).

Note: the OR-Tools Java API names can differ slightly between versions.
Follow the official examples (Nurse Scheduling) from the OR-Tools docs.


================================================================================
12. AI FEATURE 3: ATTRITION PREDICTION
================================================================================

Goal
  Predict how likely each employee is to leave and show the main reasons.

Dataset
  IBM HR Analytics Employee Attrition & Performance (Kaggle).
  File: WA_Fn-UseC_-HR-Employee-Attrition.csv (1470 rows, "Attrition"
  column = Yes/No).

Training script (train_attrition.py):

    import pandas as pd
    import joblib
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import classification_report, roc_auc_score
    from sklearn.preprocessing import LabelEncoder

    df = pd.read_csv("data/WA_Fn-UseC_-HR-Employee-Attrition.csv")

    # drop useless columns
    df = df.drop(columns=["EmployeeCount", "EmployeeNumber",
                          "Over18", "StandardHours"])
```

```
    # target
    df["Attrition"] = (df["Attrition"] == "Yes").astype(int)

    # encode text columns
    encoders = {}
    for col in df.select_dtypes(include="object").columns:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        encoders[col] = le

    X = df.drop(columns=["Attrition"])
    y = df["Attrition"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    model = RandomForestClassifier(
        n_estimators=300, class_weight="balanced", random_state=42)
    model.fit(X_train, y_train)

    print(classification_report(y_test, model.predict(X_test)))
    print("AUC:", roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))

    joblib.dump({"model": model, "encoders": encoders,
                 "columns": list(X.columns)}, "models/attrition_model.pkl")

Prediction endpoint:
  - Spring Boot sends employee features (age, job role, monthly income,
    overtime, years at company, job satisfaction, distance from home,
    etc.) as JSON.
  - Python loads the .pkl, encodes, returns risk_score (0 to 1) and the
    top 3 factors (use model.feature_importances_ as a simple
    explanation).

Mapping your data to the dataset
  Your employee table must contain similar fields. Add these columns to
  employees (or a separate employee_hr_metrics table):
    age, job_satisfaction (1-4), overtime (Y/N), years_at_company,
    years_since_last_promotion, distance_from_home, monthly_income,
    work_life_balance (1-4), job_level.
  Seed demo employees with realistic values so predictions look varied.
```

```
Display
  - Table: employee, risk %, level (Low <30%, Medium 30-60%, High >60%),
    top factors
  - Bar chart of risk distribution
  - Suggested action text (e.g. "Consider salary review" if income factor
    is high)

Report points: accuracy, precision, recall, AUC, confusion matrix, feature
importance chart. Mention the dataset is public and fictional, and that
the model is a decision-support tool, not a final decision maker.


================================================================================
13. AI FEATURE 4: HR CHATBOT (LOCAL OLLAMA)
================================================================================

Goal
  Employees ask questions in plain English and get answers based on their
  own data and company policy.

Method (RAG-lite, no vector DB needed)
  1. Employee asks a question.
  2. Spring Boot gathers context:
       - the employee's leave balance, recent attendance, upcoming shifts
       - company policy text (from the policies table; keep it short, or
         pick the most relevant policies using keyword matching)
  3. Build a prompt with this context + the question.
  4. Send to Ollama, return the reply.

Ollama endpoint:
  POST http://localhost:11434/api/chat

Request body:

    {
      "model": "llama3.2:3b",
      "stream": false,
      "messages": [
        { "role": "system",
          "content": "You are an HR assistant. Answer ONLY using the
                     context below. If the answer is not in the
                     context, say you don't know and suggest contacting
                     HR.\n\nCONTEXT:\n..." },
```

```
        { "role": "user", "content": "How many leave days do I have?" }
      ]
    }

Spring Boot call (use WebClient or RestTemplate):

    String url = "http://localhost:11434/api/chat";
    Map<String,Object> body = Map.of(
        "model", "llama3.2:3b",
        "stream", false,
        "messages", List.of(
            Map.of("role", "system", "content", systemPrompt),
            Map.of("role", "user", "content", question)));
    Map response = restTemplate.postForObject(url, body, Map.class);
    String answer = (String) ((Map) response.get("message")).get("content");

Increase the timeout (e.g. 60 seconds); a 3B model on CPU takes a few
seconds per answer.

Important details
  - Never send other employees' private data in the context.
  - Employees only get their own data; HR/Admin can ask broader questions.
  - Fallback: if Ollama is not running, show "Chatbot offline" instead of
    crashing.
  - Save chat history (optional).

Example questions to demo
  - "How many annual leave days do I have left?"
  - "What is the policy for sick leave?"
  - "What shift am I on this week?"
  - "Was I late this month?"

Cheaper fallback if the laptop is slow
  Intent matching: keyword/regex detection -> run a SQL query -> reply
  with a template sentence. Not an LLM, but works instantly. You can
  implement this first, then upgrade to Ollama.


==========================================================================
14. DESKTOP APP PACKAGING (ELECTRON)
==========================================================================

Idea
```

```
  Electron opens a desktop window that loads your React app. Spring Boot
  and the Python service run in the background.

Setup steps

  1. Create the React app:
       npm create vite@latest frontend -- --template react
       cd frontend
       npm install
       npm install axios react-router-dom recharts

  2. Add Electron:
       npm install --save-dev electron electron-builder concurrently wait-on

  3. Create electron/main.js:

       const { app, BrowserWindow } = require('electron');
       const path = require('path');

       function createWindow() {
         const win = new BrowserWindow({
           width: 1280, height: 800,
           webPreferences: { preload: path.join(__dirname, 'preload.js') }
         });
         if (process.env.NODE_ENV === 'development') {
           win.loadURL('http://localhost:5173');
         } else {
           win.loadFile(path.join(__dirname, '../dist/index.html'));
         }
       }

       app.whenReady().then(createWindow);
       app.on('window-all-closed', () => {
         if (process.platform !== 'darwin') app.quit();
       });

  4. In package.json add:
       "main": "electron/main.js",
       "scripts": {
         "dev": "concurrently \"vite\" \"wait-on http://localhost:5173 &&
                 cross-env NODE_ENV=development electron .\"",
         "build": "vite build && electron-builder"
       }
```

```
      (install cross-env too: npm install --save-dev cross-env)

  5. Webcam permission works in Electron by default for getUserMedia on
     localhost/file; test it early.

  6. Build the Windows installer:
       npm run build
     The .exe setup appears in the "dist" or "release" folder.

Starting the backend automatically from Electron (optional, advanced)
  - Use Node's child_process.spawn in main.js to run:
      java -jar backend.jar
      python -m uvicorn main:app --port 8000
    and kill them when the app closes.
  - For a final year demo, the simple start script (next section) is
    perfectly acceptable.

Packaging the backend
  - Build the jar: in /backend run  mvn clean package
    -> target/workforce-0.0.1-SNAPSHOT.jar
  - Copy it into the Electron app's resources folder (extraResources
    option in electron-builder).

Alternative
  JavaFX if you want pure Java UI. More work, less modern look. Stick
  with Electron unless your supervisor requires pure Java.


================================================================================
15. ONE-CLICK START SCRIPT
================================================================================

scripts/start-all.bat

    @echo off
    echo Starting PostgreSQL service...
    net start postgresql-x64-16

    echo Starting Ollama...
    start "" ollama serve

    echo Starting Python AI service...
    cd /d %~dp0..\ai-service
```

```
    start "AI Service" cmd /k "venv\Scripts\activate && uvicorn main:app --port 8000"

    echo Starting Spring Boot backend...
    cd /d %~dp0..\backend
    start "Backend" cmd /k "java -jar target\workforce-0.0.1-SNAPSHOT.jar"

    echo Waiting for backend...
    timeout /t 20

    echo Starting desktop app...
    cd /d %~dp0..\frontend
    start "" npm run dev

Notes
  - Run as Administrator if "net start" fails.
  - Adjust the PostgreSQL service name to match your install.
  - Ollama may already run in the background after install; if so remove
    that line.


================================================================================
16. WEEK-BY-WEEK BUILD PLAN (about 16 weeks; shrink or stretch as needed)
================================================================================

PHASE 0 - Planning (Week 1)
  - Install all software, create GitHub repo, set up folders
  - Write the problem statement, objectives, scope
  - Draw use-case diagram, ER diagram, architecture diagram
  - Get supervisor approval on scope (4 AI features, desktop app)

PHASE 1 - Backend core (Weeks 2-4)
  - Spring Boot project, PostgreSQL connection
  - Entities + repositories for all tables
  - JWT login and role security
  - CRUD: employees, departments
  - Test every API in Postman

PHASE 2 - Frontend core (Weeks 4-6)
  - React project, routing, layout (sidebar + header)
  - Login page, auth context, protected routes
  - Employees, departments pages
  - Connect to backend with Axios

```

```
PHASE 3 - Business modules (Weeks 6-8)
  - Attendance (manual first), leave, tasks, payroll
  - Dashboard with charts
  - Seed demo data (30-50 fake employees)

PHASE 4 - AI features (Weeks 9-13)
  Week 9-10:  Face recognition attendance
  Week 10-11: Shift scheduling (OR-Tools)
  Week 12:    Attrition prediction
  Week 13:    Chatbot

PHASE 5 - Desktop packaging (Week 14)
  - Electron wrapper, build installer, test on a clean PC if possible
  - Start script

PHASE 6 - Testing and polish (Week 15)
  - Bug fixing, validation messages, loading states, error handling
  - UI cleanup, consistent colours, icons
  - Performance check

PHASE 7 - Documentation and demo (Week 16)
  - Finish report, screenshots, presentation slides
  - Rehearse the demo 3+ times
  - Record a backup demo video

Rule: after each week, commit to GitHub and have something that runs.


================================================================================
17. TESTING PLAN
================================================================================

Unit testing
  - JUnit + Mockito for service classes (leave balance logic, payroll
    calculation, late detection)

API testing
  - Postman collection with all endpoints; export it for your report

Integration testing
  - Spring Boot Test with an H2 or test PostgreSQL database

AI testing
```

```
  - Face: accuracy test with multiple people/lighting conditions
  - Scheduling: check zero constraint violations, test with infeasible
    input (e.g. too few staff) and show a clear error
  - Attrition: train/test metrics (precision, recall, AUC)
  - Chatbot: list of 20 test questions with expected answers, count
    correct ones

UI testing
  - Manual test cases table: ID, scenario, steps, expected, actual, pass/fail

Security testing
  - Employee cannot access HR endpoints (expect 403)
  - Expired/invalid token is rejected
  - SQL injection safe because of JPA (mention in report)

Create a test-case table in your report; examiners like it.


========================================================================
18. REPORT / DOCUMENTATION STRUCTURE
========================================================================

  1. Abstract
  2. Introduction (background, problem statement, objectives, scope)
  3. Literature review (existing HR systems like BambooHR, Workday,
     Zoho People; AI in HR research papers)
  4. Requirement analysis (functional, non-functional, user stories)
  5. System design
       - Architecture diagram
       - Use-case diagram
       - ER diagram + table descriptions
       - Class diagram
       - Sequence diagrams (login, face check-in, roster generation)
       - UI wireframes
  6. Implementation
       - Tech stack and why chosen
       - Module-by-module explanation with screenshots
       - AI features: methodology, algorithms, datasets
  7. Testing and results
       - Test cases, AI metrics, screenshots
  8. Limitations and future work
       (anti-spoofing, mobile app, cloud deployment, payroll integration,
       bigger LLM)
```

```
   9. Conclusion
  10. References (IBM HR dataset, OR-Tools docs, face_recognition,
      Ollama, Spring docs, research papers)
  11. Appendix (sample code, API list, user manual)

Include a short USER MANUAL: how to install and use each module.

Ethics section (strong point): privacy of face data, bias in attrition
prediction, human-in-the-loop decisions, access control.


================================================================================
19. DEMO PLAN (10 to 15 minutes)
================================================================================

  1. (1 min)  Problem and idea, show the architecture diagram.
  2. (1 min)  Launch the software with the start script / installer.
  3. (2 min)  Login as Admin: dashboard, add an employee.
  4. (2 min)  Register that employee's face, then do a live face check-in
              with the webcam. Show attendance record created.
  5. (2 min)  Login as Employee: apply leave. Login as Manager: approve.
  6. (2 min)  HR: click "Generate Roster", show the weekly schedule.
  7. (2 min)  HR: attrition risk page, explain top factors.
  8. (2 min)  Employee: ask the chatbot 2-3 questions.
  9. (1 min)  Summary, limitations, future work.

Demo tips
  - Pre-load seed data so nothing is empty.
  - Close Chrome and heavy apps; the local LLM needs the RAM.
  - Warm up Ollama before the demo (ask it one question first).
  - Have a recorded video backup in case something fails.
  - Charge the laptop and carry the charger.
  - Test the webcam lighting in the demo room if possible.


================================================================================
20. RISKS AND TIPS
================================================================================

Risk: dlib/face_recognition fails to install on Windows.
  Fix: use DeepFace, or use Python 3.10/3.11 with prebuilt wheels.

Risk: local LLM is slow.
```

```
  Fix: use a 3B model or smaller, keep prompts short, or use the
  intent-matching fallback chatbot.

Risk: too much scope.
  Fix: core system + face recognition + scheduling + attrition is
  already a strong project. Chatbot is the first thing to cut.

Risk: OR-Tools model becomes infeasible.
  Fix: start with few constraints, add one at a time, test each.

Risk: team members don't know ML.
  Fix: you don't need to. Libraries do the heavy lifting. You only need
  to understand and explain: what the data is, what the model does,
  and what the results mean.

Risk: running out of time near the end.
  Fix: follow the weekly plan; always keep a working version in GitHub;
  freeze features 2 weeks before submission.

General tips
  - Use Git from day 1, commit often.
  - Validate all inputs on both frontend and backend.
  - Use DTOs, don't expose entities directly.
  - Use environment/config files for DB passwords and ports.
  - Write the report as you build, not at the end.
  - Learn enough to explain every line; examiners will ask "how does
    this work?"


================================================================================
21. WHAT TO DO RIGHT NOW (DAY 1)
================================================================================

  [ ] Install JDK 17, IntelliJ, PostgreSQL, Node.js, Python, Git
  [ ] Create a GitHub repo "workforce-system"
  [ ] Create the folder structure from section 5
  [ ] Generate the Spring Boot project at start.spring.io with:
        Spring Web, Spring Security, Spring Data JPA, PostgreSQL Driver,
        Validation, Lombok
  [ ] Create the database "workforce_db" in pgAdmin
  [ ] Configure application.properties:

        spring.datasource.url=jdbc:postgresql://localhost:5432/workforce_db
```

```
        spring.datasource.username=postgres
        spring.datasource.password=YOUR_PASSWORD
        spring.jpa.hibernate.ddl-auto=update
        spring.jpa.show-sql=true
        server.port=8080

  [ ] Run the app and confirm it starts without errors
  [ ] Create the Employee entity, repository, controller, and test
      GET/POST in Postman
  [ ] Download the IBM HR Attrition dataset from Kaggle and save it in
      ai-service/data/

Once this works, you have officially started. Everything after that is
one module at a time.

================================================================================
 Good luck with the project!
================================================================================
```